Raikou: Evidence-Grounded Understanding of SAR Imagery

A Sentinel-1 pipeline that answers questions about radar scenes from measured evidence

Author: _____ **Guide:** _____ **Date:** _____

1. Introduction

Synthetic aperture radar (SAR) is the one kind of satellite imaging that works through cloud and in darkness, which is why it matters for flood mapping, shipping, disaster response and the watch kept over remote borders. A SAR image, however, is not a photograph. Each pixel records how strongly the ground sent a microwave pulse back to the satellite, which depends on roughness, geometry and moisture rather than colour, and vision models trained on photographs misread it. Asked about a radar scene, general-purpose models, and even models fine-tuned on SAR captions, describe objects that are not there.

Raikou is a system that turns a Sentinel-1 scene into something a language model can answer questions about without inventing anything. It calibrates the raw data into physical units, measures how each part of the scene scatters the radar signal, indexes the scene for similarity search, and composes its answers only from those measurements, citing them and stating plainly what it cannot support. The work began as a retrieval-augmented generation (RAG) pipeline for SAR scenes. During the term its direction widened to a second question: whether SAR imagery can support border monitoring.

This report records the state of the project at the end of the internship term. Chapters 2 and 3 cover the data, models and architecture; Chapter 4 how the system turns pixels into evidence and evidence into answers; Chapters 5 to 7 the application, deployment and engineering record; Chapter 8 the border-monitoring question; and Chapters 9 and 10 the limitations and next steps.

1.1 Objectives

- Bring a Sentinel-1 scene in by upload or by drawing an area on a map, and process it durably.
- Convert raw radar data into calibrated, physically interpretable evidence instead of relying on a model's reading of the image.
- Make every scene searchable by visual similarity at native resolution.
- Answer plain-English questions only from recorded evidence, citing its source and refusing explicitly where evidence is absent.
- Establish what the system can and cannot contribute to border monitoring.

1.2 Status at a glance

Area	State at the end of September 2026
Processing pipeline	Ten durable, resumable stages; built, tested and run on real scenes
Scene understanding	Calibration, noise removal, scattering physics and composed answers, verified on real scenes
Web application	Deployed at www.raikou.site
Backend and GPU services	Offline since late August; the host workstation is unreachable
Vector index	Cloud cluster deleted; to be recreated and the scenes re-indexed
Border monitoring	Scoped in this report; no border area processed yet
Codebase	64 commits (12 June – 6 August 2026); about 20,800 lines of Python, 6,600 of frontend code, 16 database migrations and 128 automated tests

Table 1.1. Project status at the time of writing.

2. Data and models

2.1 Sentinel-1 imagery

All imagery comes from the European Space Agency's Sentinel-1 mission, in Interferometric Wide swath (IW) mode as Level-1 Ground Range Detected, high-resolution (GRDH) products. Sentinel-1 is a C-band radar (5.405 GHz), and these products are dual-polarised: VV (transmitted and received vertically) and VH (transmitted vertically, received horizontally). Pixel spacing is 10 m and the effective resolution about 20 × 22 m. A product is distributed as a whole frame of roughly 250 × 170 km in a SAFE archive of about 1.7 GB, which stores digital numbers rather than backscatter; the calibration and noise tables that recover physical units ship inside the same archive. GRD products carry amplitude only, with no phase, so interferometric and fully polarimetric methods are not available.

2.2 Scenes processed

The project database held eleven scene records on 28 September 2026: seven had reached ready, one had failed, one was cancelled, one was a draft and one was left mid-upload. They come from three sources (Table 2.1).

Source	What it is	Where	Size
Full SAFE archive	`S1D_IW_GRDH_1SDV_20260630T003057`: 30 June 2026, 00:30 UTC, descending pass, 38.95° incidence	Bay of Bengal coast, centred 13.79° N, 79.69° E; about 255 × 171 km	1.72 GB; 25,523 × 16,749 px
Area subsets	Renders of the Sentinel-1D acquisition of 5 August 2026; four requested, two completed	80.29–80.39° E, 13.25–13.32° N, north of Chennai	627 × 388 to 741 × 602 px at 10 m; 1.6–3.0 MB
Generic GeoTIFFs	Single rasters without a SAFE archive	Unknown: no footprint metadata	—

Table 2.1. Imagery processed so far. The full frame is the validation scene for every measurement in Chapter 4. The subsets exercise the area-of-interest path and cost 2.5–4.5 processing units each; the GeoTIFFs exercise the generic-raster fallback and, having no location, are treated as unverified throughout.

2.3 Models

Model	Role	Input	Notes
SARCLIP (GeoRS ViT-L/14), from the SARVLM project	Patch embeddings for similarity search	224 × 224 px patches at native 10 m	Contrastive image–text model trained on SARVLM-1M, over one million SAR image–text pairs; used for similarity only
reBEN ResNet-50, BigEarthNet v2.0 Sentinel-1 weights (v0.2.0)	Scene-level land-cover context	120 × 120 px windows at 10 m, [VV, VH] in dB	19 CORINE classes; macro average precision 0.628 on its test split; trained on 549,488 patches from ten European countries
SARChat vision-language model, served by vLLM	Constrained visual probe; grounded chat	Overview image and retrieved patch previews	Fine-tuned SAR captioner (InternVL2.5-2B; a Phi-3.5-vision variant on the workstation); used only for what it was measured to do reliably

Table 2.2. Models used. With all three resident, measured peak use is 9.6 GB of GPU memory, about 10 GB of RAM and 55 GB of disk.

2.4 Faults found in the data

- **The validation scene was not where the handoff said.** An early handoff document placed it at Adana on the Turkish coast. Its manifest footprint puts it on the Bay of Bengal coast between Nellore and Chennai. Location decides which models apply to a scene, so every such judgement had to be redone.

- **The archive does not store backscatter.** Measurement bands hold digital numbers. Ingestion first discarded the calibration tables and worked from 10·log10(DN + 1), a quantity with no physical scale. On the validation scene the calibration table runs from 707 at near range to 600 at far range, a 1.4 dB tilt across the swath that the uncalibrated path silently absorbed.
- **The darkest quarter of the scene was receiver noise.** The product's own noise table puts the VH noise floor at −26.8 dB (median), while the scene's VH measures −26.2 dB at its 5th percentile and −24.7 dB at its 25th. Left in, the darkest surfaces report the instrument rather than the ground, exactly where the cross-polarisation ratio carries most information.
- **The incidence angle was missing for every archive.** A file pattern matched calibration and interference annotations, which carry no incidence angle, so the field was stored empty. Corrected, it reads 38.95° for the validation scene.
- **Area subsets arrive in different units.** Sentinel Hub renders are already calibrated σ^0 in linear power, mostly below 1 (25th percentile 0.025, median 0.10 on a live subset). The conversion written for digital numbers collapses such values to within a decibel of zero: one real 224 px patch spanned only 44 of 256 grey levels in VV and 34 in VH, so every patch shown to SARCLIP and the captioner was a nearly uniform bright square. Choosing the conversion by radiometry restored 248 and 216 levels.
- **A European label space has no tropical class.** The land-cover model ranked Coniferous forest second on this tropical coast, present in 23.7% of sampled windows. Its inputs were not at fault: open water scored Marine waters at 1.00, and the scene's backscatter sits on the training mean (VV −11.9 dB against −12.6 dB). The labels simply do not describe the place.

3. System architecture

3.1 Components

Raikou has four working parts, each with one job: a browser client, an API, a durable queue and a pool of workers (Figure 3.1). State lives in Postgres, imagery and derived files in object storage, and patch vectors in a vector database.

Component	Technology	Responsibility
Client	React 18, Vite, Tailwind CSS, MapLibre GL, TanStack Query; hosted on Vercel	Projects, uploads, area selection, job status, evidence search and chat
API	FastAPI, Pydantic v2	Sign-in checks, ownership, upload plans, jobs, retrieval and chat
Queue	Postgres outbox, dispatcher, Redis Streams consumer groups	Durable hand-off of stage tasks, with leases and retries
Workers	Python; CPU and GPU classes	The ten processing stages
State	Supabase Postgres; 16 migrations	Projects, scenes, jobs, artifacts, patches, evidence records, conversations
Object storage	Amazon S3	Source archives, overviews, maps, patch previews, manifests
Vectors	Qdrant	SARCLIP embeddings, filtered by owner, project and scene
Models	SARCLIP, reBEN, SARChat on vLLM	Embeddings, land-cover context, visual probe and chat

Table 3.1. Components and their responsibilities.

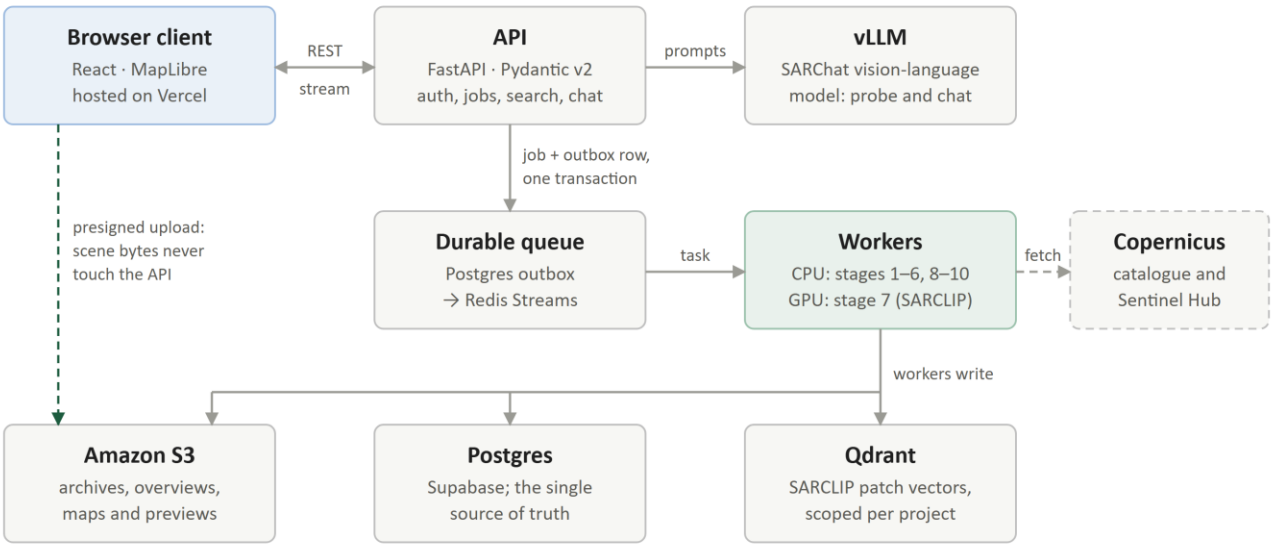


Figure 3.1. System architecture. Scene bytes go from the browser straight to object storage; the API only coordinates. Postgres is the single source of truth, and Redis exists only to wake the workers.

3.2 Design decisions

- **Scene bytes never pass through the API.** The browser uploads straight to S3 in parts using presigned URLs, and the API caps request bodies at 4 MiB. An interrupted upload can be resumed or recovered.
- **Postgres is the only source of truth.** A job and its outbox row are written in one transaction; a dispatcher moves outbox rows onto Redis Streams, which exist only to wake workers. Losing Redis delays work but never loses it.
- **Every stage can be retried on its own.** Each stage re-materialises its inputs from object storage, so worker disks are disposable. Tasks hold leases, and failures retry on a 5, 10, 20, 40 and 80 second ladder before a scene is marked failed with a reason the user can read.
- **Every read is scoped.** Queries to Postgres, Qdrant and the cache carry the owner and the project; similarity search never crosses projects.

3.3 The processing pipeline

#	Stage	Worker	What it does	Durable output
1	`fetch_source`	CPU	Downloads a frame, or renders the drawn area, from Copernicus (area-fetched scenes only)	Source archive or GeoTIFF pair
2	`validate_upload`	CPU	Checks that the source objects exist, are non-empty and have the expected layout	—
3	`extract_metadata`	CPU	Reads sensor, time, orbit, polarisation, footprint and incidence angle	Scene metadata
4	`build_vrt`	CPU	Assembles a virtual raster over the measurement bands	VRT record
5	`build_overview`	CPU	Renders the whole scene at a longest edge of 1,792 px	Overview image
6	`tile_patches`	CPU	Cuts 224 px patches at a 112 px stride, dropping any more than 40% empty	Patch rows and previews
7	`embed_patches`	GPU	Encodes every patch with SARCLIP	Embedding manifest
8	`index_vectors`	CPU	Loads the embeddings into Qdrant with ownership filters	Vector entries

#	Stage	Worker	What it does	Durable output
9	`build_evidence`	CPU	Calibrates and denoises, measures scattering, estimates land/water and land cover, renders the mechanism map	Scene record and mechanism map
10	`finalize`	CPU	Confirms that the required artifacts and vectors exist	Ready scene

Table 3.2. The ten processing stages. Uploads start at stage 2 and area fetches at stage 1. A full frame tiles into about 31,000–33,000 patches; an area subset into about 20 (a 7 km box) to about 440 (the 25 km maximum).

3.4 Two ways in

- **Upload an archive.** The user uploads a SAFE .zip. The upload is multipart and resumable, the file is checked in the browser before it starts, and the server verifies it again before processing: its size against the upload plan, a valid zip with no unsafe paths, and a SAFE manifest.
- **Draw an area.** The user draws a box on the map, and the server searches the Copernicus Data Space catalogue for IW GRDH dual-polarisation products that cover it. That product lock is enforced three times: by the query, by a database constraint and when the file arrives, which must hold both polarisations and every calibration and noise table. The server then renders only the drawn box through the Sentinel Hub Process API, orthorectified, calibrated to σ^0, at 10 m pixels and pinned to the single acquisition chosen. A 24 km box arrives as 39 MB in seconds instead of a 1.7 GB frame; the subsets used so far were 1.6–3.0 MB.

A box is capped at 25 km a side because the pixel size is never coarsened to fit a larger area. The land-cover model is trained on 1.2 km windows at 10 m; at 40 m the same window would cover 4.8 km, and the model would return confident nonsense rather than a weaker answer.

4. From pixels to evidence to answers

The central design choice is that no model is ever asked to look at the image and say what is there. Understanding is built in layers, and nothing is said about a scene until a layer has measured it (Figure 4.1).

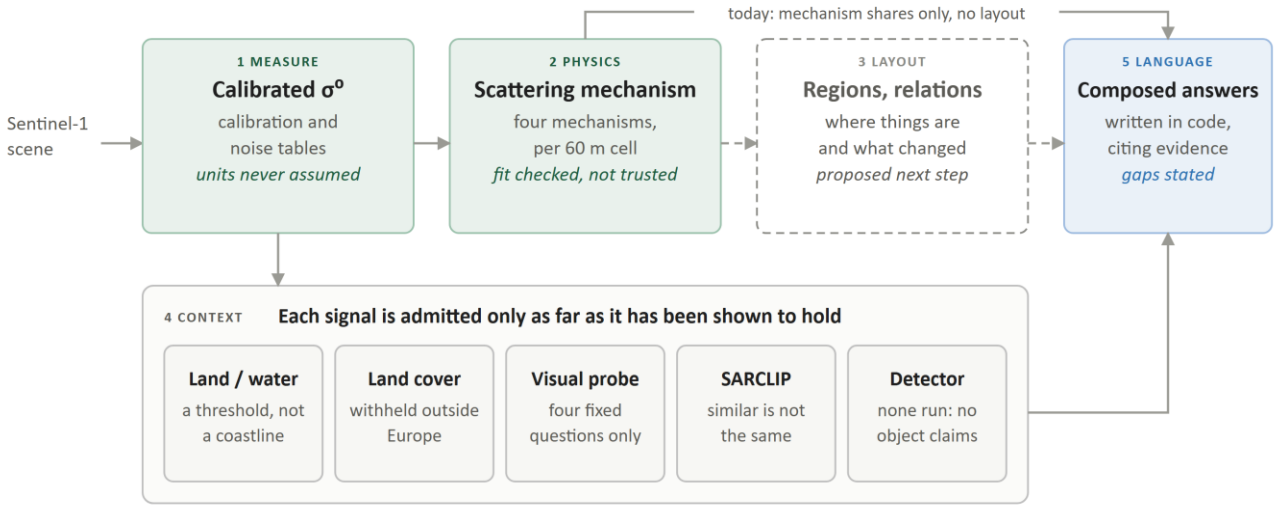


Figure 4.1. How Raikou builds its understanding of a scene. Each layer is measured before anything is said, and each context signal carries its own gate. The dashed layer, spatial layout, does not exist yet: today the language layer receives how much of each mechanism a scene holds, but not where (Chapter 10).

4.1 Measuring: calibrated backscatter

For an uploaded archive, digital numbers are converted to σ^0, the radar backscatter coefficient, with the archive's own calibration table ($\sigma^0 = DN^2 / A^2$, A interpolated bilinearly to every pixel), after subtracting the

thermal-noise estimate from its noise table. The noise model has two terms, a range table and a per-sub-swath azimuth scaling (1.00–1.13 on the validation scene, at most 0.53 dB), and both pass through the same interpolation as the calibration, so no residual follows the tables' node spacing. Windows are calibrated on demand rather than materialising a calibrated raster, which for the 25,523 × 16,749 px two-band frame would take 3.3 GB. Calibrated values land in the expected −20 to −6 dB band. Area subsets arrive already calibrated, and which form each file holds is recorded and branched on explicitly.

4.2 Physics: scattering mechanism

Instead of naming land cover, the system measures how each part of the scene scatters the radar signal, because that is set by surface geometry and holds in any region: a canopy depolarises the return in Finland and in a mangrove alike. Four mechanisms are separated using VV backscatter, the cross-polarisation ratio VH/VV, and the radar vegetation index RVI = 4·VH / (VV + VH) (Table 4.1).

Mechanism	Physical cause	Rule	Typical surfaces
Smooth surface	A smooth interface reflects the pulse away, with little depolarisation	VV and VH/VV both below the scene's fitted water levels	Open water, wet flats, dry sand
Double bounce	A ground–wall corner returns the pulse along its own path	VV ≥ −6 dB and VH/VV ≤ −8 dB	Buildings, ships, infrastructure
Volume	Repeated scattering inside a canopy depolarises the return	RVI ≥ 0.55	Forest, plantation, tall crops, mangrove
Rough surface	A moderate, weakly depolarised return	Everything else	Bare or sparsely vegetated ground

Table 4.1. Scattering mechanisms. Rules are tested from the most specific to the most general. The water levels are fitted per scene; the other levels are fixed physical values.

Thresholds are fitted, then checked. The water boundary is fitted to each scene: its brightness limit is the scene's 60th percentile of VV, and its ratio limit an Otsu split of the cross-polarisation ratio (Otsu's method picks the threshold that best separates a histogram into two populations). Textbook C-band levels had placed 0.1% of the validation scene in the water class against an independent estimate of 36%, because a monsoon sea is rough and never darkens to calm-water levels. On the validation scene the split fell at −12.22 dB and separated two populations an order of magnitude apart in depolarisation: RVI 0.07 below it and 0.70 above.

Otsu always returns a threshold, even through a single population, and a small area can hold only one surface. On a 7 × 6 km urban subset the fitted water level (−5.14 dB) came out brighter than the built-up level (−6.00 dB), and a quarter of the city was labelled open water. A fit is now rejected when the water level crosses the built-up level, when VV spans less than 8 dB between its 5th and 95th percentiles, or when fewer than 200 windows were sampled. Fixed conservative levels (VV ≤ −15 dB and VH/VV ≤ −12 dB for water) are then used, and the record states which of the two produced its numbers.

From a share to a map. Scattering is measured on a sample of 120 px windows for the scene record, and densely for the mechanism map. Speckle is multiplicative, so the map averages power, not decibels, over 6 × 6 pixel blocks, giving 36 looks per 60 m cell, and then applies a 3 × 3 majority filter that removes isolated cells without inventing edges. The rule is arithmetic on two numbers per cell, so the whole raster is classified in one pass with no learned model (Figure 4.2).

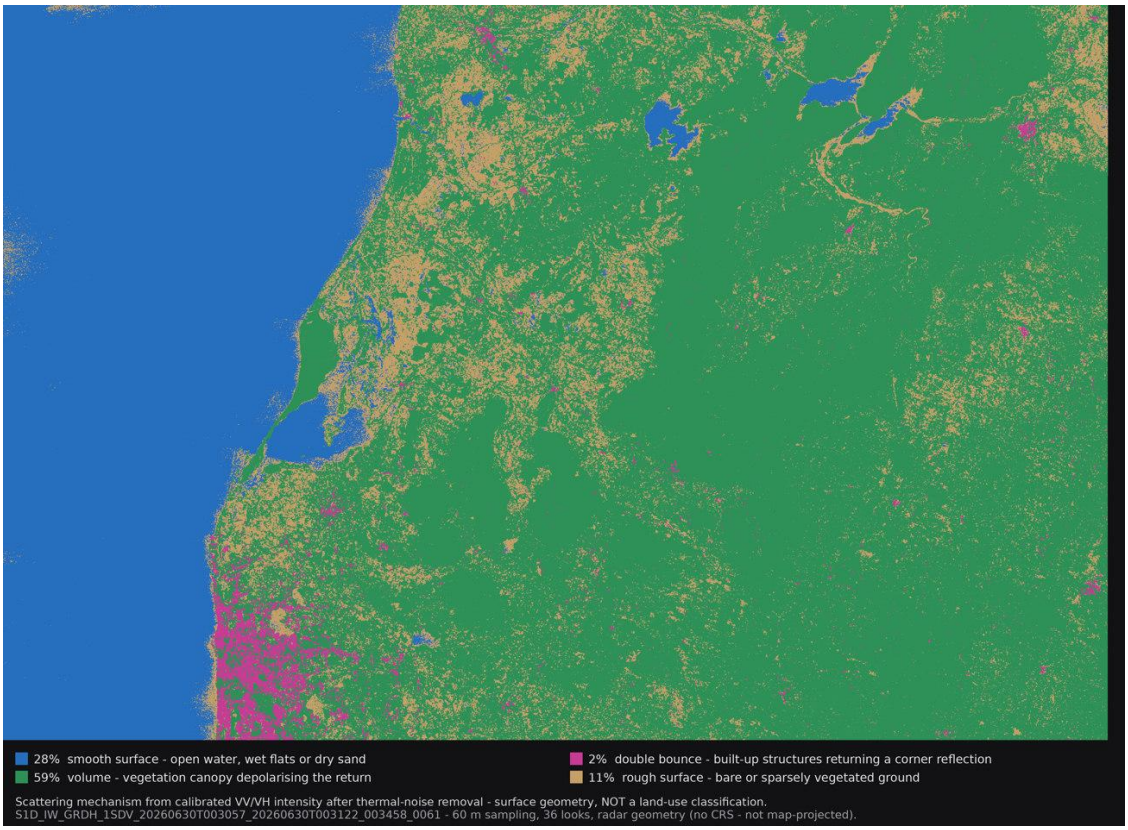


Figure 4.2. Scattering-mechanism map of the validation scene, as produced by the pipeline (60 m cells, 36 looks). The legend and the statement that this is surface geometry, not land use, are burned into the image so they survive being copied elsewhere. The map is in radar geometry rather than a map projection, so it cannot be overlaid on a basemap; the open sea is the blue field on the left.

Mechanism	Share of sampled windows	Median VV (dB)	Median VH/VV (dB)	Share of dense map
Volume	62.7%	−10.2	−6.7	59%
Smooth surface	26.5%	−14.0	−17.8	28%
Rough surface	8.3%	−10.5	−8.5	11%
Double bounce	2.4%	−2.8	−11.3	2%

Table 4.2. Scattering on the validation scene. Of 2,948 sampled windows, 35 whose VH sat on the noise floor were discarded rather than averaged in. The dense map covers 98.5% of the raster, and answers quote its shares so that the text and the map legend agree. An independent brightness-only estimate puts water at 36.4%, above the 26.5% smooth-surface share, as expected of a method that also counts dark, dry ground as water.

4.3 Context signals, each behind a gate

Four further signals add context. Each is admitted only as far as it has been shown to hold.

- **Land/water estimate.** A brightness threshold splits the scene into land-like and water-like backscatter. It is reported as a share with a label (land-dominant, water-dominant or indeterminate) and described in the answer as “a brightness threshold, not a coastline map”.
- **Land cover, withheld outside its domain.** The reBEN model scores 19 CORINE classes on 120 px windows, behind two guards. A high-precision operating point ($p \geq 0.95$ rather than the training threshold of 0.5) keeps the classes the model is sure of, with Marine waters falling only from 0.312 to 0.284 of windows, and collapses the diffuse ones: Coniferous forest from 0.237 to 0.034 and Transitional woodland from 0.185 to 0.003. A geographic gate then withholds the classes unless the scene lies inside the training footprint (longitude −31.3° to 31.6°, latitude 36.0° to 70.1°); a location that cannot be verified fails closed. The validation scene lies outside, so its land cover is kept for review but never shown in chat.

- **A visual probe limited to what the model can see.** Asked to describe a scene, the SAR-tuned captioner answered “there is a ship in the bottom left portion” for a pure black image, for random noise and for real patches alike. Asked a narrow question with its answers listed, it reads the pixels correctly. It is therefore used only as a probe with four fixed questions (overall brightness, whether bright and dark regions coexist, texture, and the character of boundaries) in a single-image prompt, and its answers are folded into a sentence in code.
- **Similarity, never identity.** SARCLIP embeddings let a user find patches that look alike anywhere in a project. A neighbour is reported as visually similar, never as proof of what it contains.

4.4 Language: answers composed from evidence

Questions are routed by the evidence they need (Table 4.3). Scene-level questions are answered from the scene record before any similarity search runs, so they cannot be swayed by whatever retrieval happens to return.

Question	Answered from	Behaviour
Describe the scene	The scene record, composed in code	Acquisition and location; land/water; detector status; scattering; land cover if in domain; probe impressions; retrieval note
What kind of place is this?	Land cover in domain, otherwise scattering, otherwise the brightness estimate	Uses the best basis valid for the location, and names it
How many, is there, where is (objects)	Validated detector evidence only	None exists, so the answer states that no object can be confirmed
Show me evidence	Retrieved patches	Similar patches with their scores, cited
Anything else	The language model over the retrieved evidence	Cites its sources; returns “insufficient evidence” when support is missing

Table 4.3. How questions are answered. Object claims can enter only through a validated detector file (schema raikou.detector.v1). No detector has been run, so no object claim can be made.

The scene description is assembled from stored fields rather than generated. For the validation scene it opens:

> SENTINEL-1D scene, acquired 2026-06-30 00:30 UTC, descending pass, VV/VH polarisation. Centred 13.79N, 79.69E, covering roughly 255 x 171 km, imaged at a 39° incidence angle.

It goes on to give the land/water share, the scattering shares with a note that they describe a mechanism and not a land use, and the probe’s impressions, and it always carries the sentence:

> No detector has been run on this scene, so I cannot confirm any object, count or position. That is an absence of evidence, not evidence of absence.

This replaced an earlier design in which the captioner narrated the scene. Every description then read “there is a ship”, whatever the scene, for two reasons: the overview was downscaled 24.6 times, so a 200 m vessel occupied less than one pixel, and the model is an object captioner that names objects whatever it is shown.

5. The web application

The client at www.raikou.site is a React single-page application, deployed on Vercel from the repository’s main branch. A session follows five steps.

- **Sign in and open a project.** Authentication is handled by Supabase, and every project, scene and conversation belongs to one user.

- **Add a scene.** Upload an archive, or draw a box on an OpenStreetMap basemap, choose an acquisition from the catalogue results and let the server fetch it. The box stays visible while it is being drawn and can be corrected before fetching.
- **Follow processing.** A job card shows the current stage and progress. Processing runs as a background job, so the tab can be closed at any point.
- **Read the scene.** The workspace puts the overview, the mechanism map, patch previews and metadata first, with artifacts and stage details behind a disclosure.
- **Ask questions.** Evidence search across the project, and a chat panel that streams grounded answers with collapsible evidence cards.

Module	Purpose	Lines
`SceneUploadPanel`	Multipart, resumable archive upload with recovery	966
`AoiMapPicker`	Drawing and correcting the area on the map	636
`ProjectWorkspace`	Scene list, scene view, artifacts and patches	626
`CopernicusSearchPanel`	Catalogue search and choice of acquisition	436
`multipartUpload`	Part planning, presigned URLs and retries	436
`GroundedChatPanel`	Streaming chat with evidence cards	253

Table 5.1. The largest client modules, out of 6,588 lines across 24 source files.

6. Deployment and infrastructure

6.1 Where the system has run

Period	Host	Outcome
July to early August 2026	AWS EC2 g5.xlarge spot instance (NVIDIA A10G, 24 GB) in the Mumbai region	Worked, but was interrupted repeatedly and changed public address on each restart
August 2026	Lab GPU workstation (NVIDIA RTX A4000, 16 GB): five Windows scheduled tasks, vLLM in WSL2, reachable only over a private Tailscale network	Free and stable, but not publicly reachable
Throughout	Vercel (client), Supabase (Postgres and sign-in), Amazon S3 (storage), Qdrant Cloud (vectors)	Managed services

Table 6.1. Hosting history.

Both machines ran all three models on one GPU. On the 16 GB workstation card, vLLM’s share of memory was lowered from 0.80 to 0.65: at 0.80 it held about 14.4 GB, and the first real embedding job failed with 255 MB free, because SARCLIP needs about 4.6 GB beside it.

6.2 State at the time of writing

Component	State	Consequence
Web client (Vercel)	Online	Loads, but has no backend to call
API, workers and vLLM (workstation)	Offline; host last seen on the private network around 29 August	No processing, search or chat
api.raikou.site	Still points at the retired EC2 address	Must be re-pointed after redeployment
Qdrant Cloud cluster	Deleted	Patch vectors lost; a new cluster is needed and scenes must be re-indexed
Supabase	Running; answered on 28 September	State intact, but see the network risk below
Amazon S3	Not re-checked at the time of writing	Archives and derived artifacts expected to be retained

Table 6.2. State of each component on 28–29 September 2026.

Supabase is not reachable from every Indian network. On 29 September supabase.co could not be reached from a home broadband connection: the DNS lookup was refused and a direct connection timed out, although it had answered from the campus network the day before. In February 2026 several Indian ISPs, including Jio, Airtel and ACT, blocked supabase.co, and access was restored in March. Because the client signs users in against Supabase directly from the browser, a demonstration must be tested on the network where it will be given; a custom domain or a proxy in front of Supabase would remove the dependency.

6.3 Plan to restore it

A USD 50 AWS credit is available. A g5.xlarge costs about USD 1 an hour on demand (the us-east-1 list price; other regions differ slightly), so running continuously the credit would last about two days. The plan is to run it only for processing, for recording a demonstration and for the presentation itself, about 10–15 hours in all, and to stop it otherwise. The steps are to confirm the account's GPU instance quota; check for resources left from the earlier EC2 deployment (an unattached Elastic IP is billed every month); create a new Qdrant cluster, whose collection the code creates on first use; redeploy with the existing deployment scripts; re-point api.raikou.site; and reprocess the scenes.

7. Engineering record

7.1 Automated tests

Test module	What it checks	Tests
`test_copernicus_client`	Catalogue search contract, product lookup, throttling and error mapping, against recorded responses	28
`test_land_cover_calibration`	Calibration-table parsing and interpolation, σ^0 against the ESA definition, land-cover inputs	19
`test_scene_geography`	Footprint parsing, ground extent, hemispheres, incidence angle, sentence rendering	16
`test_scattering_map`	Label indexing, rule precedence, majority filter, no-data handling, rendering	14
`test_acquisition_archive_layout`	Only complete dual-polarisation products with every table are accepted	11
`test_scattering`	Multi-looking in linear power, ratio and RVI, noise-limited windows, rule order	10

Test module	What it checks	Tests
`test_noise_removal`	Noise-table parsing, per-swath azimuth scaling, clamping at the floor	8
`test_scene_chat_policy`	Question routing; a missing detection is never turned into an absence	8
`test_upload_archive_validation`	Archive size, manifest, magic bytes, entry count and path traversal	7
`test_job_stage_enum_parity`	Stage and status names agree between code and database	3
`test_generic_tiff_visuals`	Ordinary RGB rasters are not treated as radar power	2
`test_release_hardening`	Safe request identifiers and metric labels	2

Table 7.1. Automated tests: 128 across 12 modules.

7.2 Faults found in live operation

Much of the engineering was finding out why something that looked correct was not. Table 7.2 lists the faults found while running real scenes, each with the measurement that exposed it.

What was observed	Cause	Fix
A worker froze for over 100 minutes mid-scene, and later for over 20 minutes inside a single-row insert, with no error	Database connections without timeouts, and a fresh connection per patch against the pooler	Connect timeout, 15 s statement timeout and TCP keepalives; one connection per task. A hang now surfaces as a retryable error
Tiling a full frame took close to four hours	Workers in Mumbai and the database pooler in Sydney, about 167 ms apart; three round trips per patch capped tiling near 2 patches a second	500-row batched upserts: 2.2 → 200 patches a second, and about 94,000 → 63 round trips for a 31,000-patch scene
Every embedding task failed with “No CUDA GPUs are available”	The GPU worker was started with device index 1 on a single-GPU machine	Instance number documented as the CUDA device index; deployments target device 0
Grounded chat failed on every ready scene	vLLM served a 4,096-token context; prompts needed about 9,650 because each image expands into up to twelve tiles	Context raised to 16,384; images are shed and the request retried if a prompt still overflows
Every scene description read “there is a ship”	An overview downscaled 24.6 times, and an object captioner that narrates whatever it is shown	Descriptions composed in code; captioner limited to a four-question probe; overview raised to 1,792 px
A scene reached ready with its scattering, map and land cover silently missing	A single-polarisation product was accepted	Product type and polarisation locked in the query, in a database constraint and on arrival
Every area fetch failed with “unexpected response”	The product identifier was quoted in the catalogue lookup, which the catalogue rejected	Lookup by key; provider error bodies kept for diagnosis
A brief provider outage killed a job in 13 seconds	All five retries were spent inside the provider’s bad minute	In-request retries at 4 s and 8 s on server errors and throttling, before the durable ladder
A deploy deleted 13 GB of model weights	`rsync --delete` ran over a directory that is not in the repository	Weights excluded; restored from the deploy’s own backup using hard links
The drawn area vanished from the map	MapLibre simplified the small rectangle out of its tiles	Simplification disabled on the area layers

Table 7.2. Faults found in live operation, with figures as measured at the time. Faults in the data are listed in Section 2.4.

7.3 Timeline

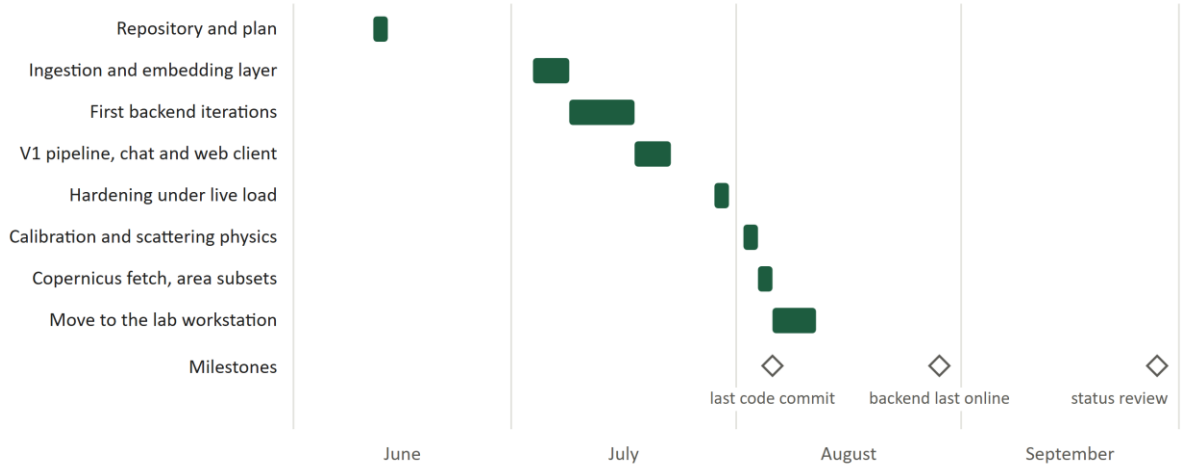


Figure 7.1. Project timeline from the repository history. Code work ran from 12 June to 6 August 2026 and was followed by the move to the lab workstation; the backend was last online around 29 August.

8. Border monitoring: what the system offers and what it would need

Border monitoring asks for repeated observation of wide, remote areas in any weather: what is present along a boundary, and what has changed since the last look. Radar suits that brief, because it sees through monsoon cloud and in darkness, and Sentinel-1 revisits the same ground every 6 to 12 days at no cost. This chapter measures the current system against that brief. No border area has been processed yet, so it is an assessment of capability, not a result.

8.1 What the current system already contributes

- **It can look anywhere.** The area path fetches any box up to 25 km a side from the public archive, calibrated and orthorectified at 10 m, in seconds.
- **Its physics travels.** Scattering mechanism needs no regional training data, so it applies unchanged to desert, delta or valley, where European land-cover labels do not.
- **It does not over-claim.** Answers cite measurements and state what is missing. In monitoring, where an unfounded alert has a real cost, a system that can say “insufficient evidence” is a requirement, not a weakness.
- **It is durable.** Long downloads and GPU stages survive restarts and interrupted machines.

8.2 What it cannot yet do

- **One date at a time.** Each scene is analysed on its own. Monitoring is about change, and the pipeline has no way to relate two acquisitions of the same area.
- **What, but not where.** The description reports how much of each mechanism a scene contains, not how it is arranged. The mechanism map holds that information, but the language layer never reads it, so the system cannot say that a settlement sits beside a river or that a new cluster has appeared near a boundary.
- **Landscape scale only.** At 60 m cells the map resolves settlements, water bodies, rivers and fields, but not roads, fences or vehicles.
- **Full frames are not map-aligned.** Without terrain correction a whole frame stays in radar geometry; only area subsets are map-projected.

8.3 What the sensor allows

Within reach of Sentinel-1	Beyond it, and why
Settlements and new construction, through double bounce	Individual vehicles or people: a car covers about 2% of a 20 × 22 m resolution cell
Water bodies, flooding and changes in river channels	Small boats among the clutter of creeks and estuaries
Land clearing, earthworks and new tracks at landscape scale	Activity under dense canopy, since C-band returns mostly from the canopy top
Large vessels at sea	Steep slopes facing towards or away from the sensor, which are lost to layover and shadow
Change measured over weeks	Anything that happens between passes: a 6–12 day revisit rules out real-time use

Table 8.1. What Sentinel-1 can and cannot contribute to border monitoring.

The conclusion for this project is that open radar can support border monitoring as a way of understanding what is present and what has changed across wide areas, directing attention to where it is needed, rather than as a way of identifying individual objects, which needs higher-resolution, tasked imagery. The next steps (Chapter 10) are designed for that role.

9. Limitations

- **Validated on one full frame.** Every calibration, scattering and land-cover figure in this report comes from a single tropical coastal scene in the monsoon season. Other landscapes and seasons have not been measured.
- **No ground truth yet.** Scattering shares have been compared with an independent brightness estimate, not with a reference map, and composed answers have not been scored.
- **The small-area check rests on mis-scaled data.** The urban subset that motivated rejecting degenerate fits was measured before the unit fault in Section 2.4 was found. The check stays, because a small area genuinely can hold only one surface, but it should be re-measured on corrected subsets.
- **Mechanism is not land use.** Paddy, mangrove and plantation all scatter by volume; open water, wet mud and dry sand all appear as smooth surface.
- **No land cover for Indian scenes.** The only land-cover model is withheld outside Europe. Scattering fills the gap but answers a different question.
- **The captioner gives impressions only.** Its object claims are unreliable and are discarded by design.
- **The documentation names two models.** The README and the default configuration name InternVL2.5-2B, while the workstation served a Phi-3.5-vision SARChat model; this must be reconciled before the system is described externally.
- **The live system is offline and its vector index is lost** (Chapter 6).

10. Next steps

The next steps keep the same design, understanding built from measured layers, and point it at the border question.

- **Spatial layout as a new layer.** Turn the mechanism map into a scene graph: regions (connected areas of one mechanism above a stated minimum size), their properties (area, position, shape, whether they touch the scene edge) and their relations (adjacent, enclosed, direction, distance). The scene description

gains a layout paragraph, chat can answer “where” questions, and each sentence cites a region that the map can highlight. The language model reasons over this structure, never over pixels.

- **Change between two dates.** Fetch the same area on two acquisitions, build a scene graph for each and describe the difference in words: a region that grew, shrank, appeared or changed mechanism.
- **A measured evaluation.** Score the regions against ESA WorldCover, a free global land-cover map at 10 m: how often double bounce falls on built-up land, smooth surface on water, and volume on trees or cropland. This gives the understanding layer its first quantitative measure, and tests on Indian ground the claim that scattering physics travels where European labels do not. WorldCover is itself partly derived from Sentinel-1, so it is a reference rather than fully independent ground truth.
- **A border test area.** A flat, low-clutter stretch of the Rajasthan border or the Rann of Kutch: flat ground avoids layover and shadow, a dark desert background makes structures stand out, and the dry season allows a cloud-free optical check.
- **Restore the live system and record a demonstration** (Section 6.3), so the presentation does not depend on a live connection.

Step	Estimated effort	Needs the GPU
Scene graph and layout paragraph	About 2 days	No
WorldCover evaluation	About 1 day	No
Two-date comparison on a border area	About 1 day	No
Redeploy, reprocess and record	About half a day	Yes: 10–15 hours in total

Table 10.1. Proposed work, in order.

Appendix A. Technology stack

Layer	Choice
Client	React 18, Vite, Tailwind CSS, MapLibre GL, TanStack Query, Supabase JS; hosted on Vercel
API	FastAPI, Pydantic v2, streamed (NDJSON) answers
Queue and workers	Postgres outbox, Redis Streams consumer groups, Postgres task leases
Data	Supabase Postgres with authentication, Amazon S3, Qdrant
Raster processing	GDAL virtual rasters through rasterio, NumPy
Models	SARCLIP GeoRS ViT-L/14 (open_clip), reBEN ResNet-50, SARChat served by vLLM
Imagery	Copernicus Data Space: OData catalogue and Sentinel Hub Process API
Operations	systemd units and nginx on EC2; Windows scheduled tasks and WSL2 on the workstation; Prometheus-format metrics

Table A.1. Technology stack.

Appendix B. Repository layout

Path	Contents
`backend/app/api/routes`	HTTP endpoints: projects, uploads, acquisitions, processing, evidence and chat
`backend/app/workers`	Dispatcher, runner and the ten stage handlers
`backend/app/services/processing`	Scattering, mechanism map, radiometry, scene record, chat policy
`backend/app/services/ingestion`	Calibration and noise tables, file ingestion

Path	Contents
`backend/app/services/models`	SARCLIP encoder and land-cover model
`backend/app/services/acquisitions`	Copernicus catalogue client and area renders
`backend/tests`	The 128 automated tests
`frontend/src`	The React client
`supabase/migrations`	16 schema migrations, applied in filename order
`deployment`	nginx configuration, systemd units and the deploy script

Table B.1. Where things live in the repository (github.com/Trivo121/Raikou).

References

[1] European Space Agency. Sentinel-1 SAR User Guide: Level-1 Ground Range Detected products. Copernicus Sentinel Online.

[2] Q. Ma, X. Lu, W. Liu, P. Duan, X. Kang and S. Li. SARVLM: A Vision Language Foundation Model for Semantic Understanding in SAR Imagery. arXiv:2510.22665, 2026.

[3] K. N. Clasen, L. W. Hackel, T. Burgert, G. Sumbul, B. Demir and V. Markl. reBEN: Refined BigEarthNet Dataset for Remote Sensing Image Analysis. arXiv:2407.03653, 2024.

[4] N. Otsu. A threshold selection method from gray-level histograms. IEEE Transactions on Systems, Man, and Cybernetics, 9(1), 62–66, 1979.

[5] D. Zanaga et al. ESA WorldCover 10 m 2021 v200. Zenodo, 2022. doi:10.5281/zenodo.7254221.

[6] MediaNama. Supabase flags possible ISP-level block affecting JioFiber users in India. February 2026.

[7] Analytics India Magazine. Supabase appeals to Ashwini Vaishnaw after access on Jio, Airtel, ACT gets blocked. February 2026.